DOCES ARTESANAIS — APLICAÇÃO WEB

Documentação da Situação-Problema 2

Projeto Integrador Transdisciplinar em Engenharia de Software II. Implementação desenvolvida no Base44 com interface React/TanStack Start e funções JavaScript no servidor.

Identificação do projeto

- Título de entrega: Cupcake Gourmet — versão publicada Doce Ofício.
- Versão: snapshot do sistema Base44 publicado em 11 de novembro de 2026.
- Repositório: PedroBorgesEng/desktop-tutorial, pasta projetos/pite-ii-cupcake-gourmet/base44-site.
- Site: https://cupcake-gourmet-pite-ii-6e0cb87a.base44.app
- Origem dos requisitos acadêmicos: PITESoftwareII.pdf, páginas 32 a 34 e 38.

Objetivo

Demonstrar uma solução full-stack para pedidos de produtos artesanais, contemplando navegação de clientes, carrinho, solicitação de reserva, processamento de preços no servidor, persistência de pedidos e painel de gestão.

Limites da demonstração

Esta implementação não processa cobranças reais. O cliente registra um pedido para pagamento combinado na retirada ou entrega. O backend e a autenticação utilizam os serviços Base44.

1. REQUISITOS E ESCOPO

Usuários e jornadas

- Cliente: explora catálogo, filtra produtos, visualiza detalhes, monta carrinho e confirma uma reserva com contato, data e retirada/entrega.
- Administrador: autentica-se para gerir produtos, disponibilidade e acompanhar estados dos pedidos.

Requisitos funcionais implementados

- RF01 — Exibir catálogo de produtos disponíveis.
- RF02 — Mostrar detalhes, descrições e imagens dos produtos.
- RF03 — Incluir, alterar quantidades e remover itens do carrinho.
- RF04 — Coletar nome, telefone e informações para retirada/entrega.
- RF05 — Validar o pedido no servidor e recalcular valores a partir do catálogo.
- RF06 — Criar registro persistente de reserva e apresentar seu resultado.
- RF07 — Permitir consulta ao estado de uma reserva.
- RF08 — Administrar produtos, preços, disponibilidade e pedidos.

Requisitos não funcionais

- RNF01 — Interface responsiva, organizada por rotas e componentes.
- RNF02 — Tratamento de campos inválidos e erros de requisição.
- RNF03 — Segurança de escritas no catálogo e pedidos por permissões.
- RNF04 — Separação entre apresentação, domínio e persistência.
- RNF05 — Código versionado e documentação disponível em PDF.

Premissas

Trata-se de aplicação acadêmica. O fluxo de reserva não constitui pedido faturado e não substitui controles fiscais, meios de pagamento, proteção de dados ou homologação de produção.

2. ARQUITETURA DE SOFTWARE

Visão em camadas

- View: React com TanStack Start, componentes responsivos em src/components/site e componentes administrativos.
- Controller: rotas em src/routes, incluindo /api/catalogo e /api/pedidos, além de server functions.
- Model: entidades Produto, Pedido e User da plataforma Base44 e validação com Zod.
- Service: src/lib/reservas.server.js concentra validações e o processamento das reservas.

A arquitetura adota separação de responsabilidades similar ao MVC recomendado na disciplina. A interface nunca define preços definitivos: os valores são relidos no servidor.

Rotas principais

GET / — página inicial e catálogo

GET /produto/:id — apresentação detalhada

GET /carrinho — montagem e revisão do carrinho

GET /pedido/:id — consulta de reserva

GET /admin — painel administrativo

GET /admin/produtos — manutenção do catálogo

GET /admin/pedidos — gestão de reservas

GET /api/catalogo — dados públicos do catálogo

POST /api/pedidos — criação de reserva validada

Organização de fontes

O site utiliza package.json, src/routes, src/components, src/lib e base44/entities. O GitHub inclui uma cópia integral dos 137 arquivos-texto exportados do projeto, exceto arquivos de instruções internos do ambiente.

3. MODELO DE DADOS E BACK-END

Entidade Produto

A entidade armazena nome, categoria, preço, resumo, descrição, ingredientes, porção, imagem, disponibilidade e sinalização de destaque. As categorias do exemplo abrangem bolos, tortas, doces e cookies.

Entidade Pedido

A reserva contém código, nome e contato do cliente, modalidade e endereço de entrega, data desejada, período, observações, itens, subtotal, taxa de entrega, total, estado e forma de pagamento combinada.

Entidade User

A plataforma mantém a identificação dos usuários e o papel (admin ou user). As políticas de dados restringem operações de escrita e leitura conforme o papel, com criação de reservas feita por operação de servidor.

Fluxo de uma reserva

- 1. O usuário escolhe produtos e quantidades no carrinho.
- 2. A interface envia dados do pedido para POST /api/pedidos.
- 3. O servidor valida campos com Zod e rejeita solicitações inválidas.
- 4. O servidor consulta os produtos disponíveis e recalcula valores.
- 5. O processamento grava um pedido e retorna resposta ao cliente.
- 6. O administrador acompanha o pedido em seu painel.

Tratamento de erros

Exemplos observáveis no código: rejeição de JSON malformado, validação de nome/telefone, limitações de quantidade, restrições de tipo de entrega e verificação de permissão administrativa.

4. FRONT-END E TESTES

Interface do usuário

O front-end apresenta capa da loja, seções de catálogo e ateliê, filtro por produtos, cartões de produtos, carrinho e telas de acompanhamento de reservas. Componentes reutilizáveis permitem manter aparência e comportamentos consistentes.

Interface administrativa

O painel organiza indicadores, cadastro e edição de produtos, controle de disponibilidade e acompanhamento de reservas. Seu acesso depende de autenticação e privilégios.

Verificação de software

- Build da versão Base44: o construtor registrou compilação após corrigir uma importação ausente em src/routes/_authed/admin/produtos.jsx.
- Testes de back-end: o histórico do Base44 registrou execuções bem-sucedidas de funções de teste do construtor.
- Testes automatizados da implementação Python alternativa: 14 testes de API e 5 testes de JavaScript aprovados, relatados separadamente.
- Limitação: não há evidência de execução completa de testes E2E no navegador para a versão Base44 no momento deste snapshot.

Critérios de aceitação recomendados

- O catálogo deve apresentar produtos disponíveis sem erro de renderização.
- O carrinho deve atualizar quantidades e valores exibidos.
- O servidor deve rejeitar requisição inválida com mensagem controlada.
- Um pedido válido deve ser persistido e localizado no painel.
- A edição de produtos deve ficar restrita ao administrador.

5. PUBLICAÇÃO E UTILIZAÇÃO

Acessar a versão publicada

A loja é acessível em https://cupcake-gourmet-pite-ii-6e0cb87a.base44.app. A aplicação publica a interface e os serviços em infraestrutura gerenciada Base44.

Reproduzir o código

- 1. Baixar ou clonar o repositório PedroBorgesEng/desktop-tutorial.
- 2. Entrar em projetos/pite-ii-cupcake-gourmet/base44-site.
- 3. Ler o README e conferir as dependências do package.json.
- 4. Conectar conta/projeto Base44 compatível para entidades, autenticação e dados.
- 5. Instalar dependências e executar o comando de desenvolvimento informado pelo projeto.

Sem configuração Base44 compatível, a interface local não reproduz automaticamente o banco de dados ou as integrações de autenticação da aplicação publicada.

Relação com a implementação Python

A pasta projetos/pite-ii-cupcake-gourmet/app pertence a uma implementação alternativa baseada em FastAPI, SQLite e testes automatizados. Ela é independente do código do site Base44, mantido em base44-site.

Controle de versão

A exportação dos arquivos Base44 foi realizada manualmente. Por restrição do plano atual, a plataforma não tem sincronização automática com o repositório GitHub. Novas alterações no editor devem ser exportadas novamente.

Conclusão

A Situação-Problema 2 apresenta uma aplicação full-stack com interface funcional, servidor, entidades persistentes e tratamento básico de erros, acompanhada de documentação e registros explícitos de validação e de limitações.